COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/31/2026	**DATE SUBMITTED:**	8/5/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL

Test the stack STL operations through the given code below. Provide screenshot per operation, your observations and any other remarks in the output section of your activity template.

CODE:

Braganza_Seatwork4_1.cpp

ILOA > Braganza_Seatwork4_1.cpp > ...

```cpp
#include <iostream>
#include <stack>

int main() {

    // Create an object - creating the stack object
    std::stack<char> stack1;

    // isEmpty - checking if the stack is empty
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;

    // Push - Adding elements to the stack
    stack1.push('R');
    std::cout << "Top element: " << stack1.top() << std::endl;
    stack1.push('A');
    std::cout << "Top element: " << stack1.top() << std::endl;
    stack1.push('L');
    std::cout << "Top element: " << stack1.top() << std::endl;
    stack1.push('P');
    std::cout << "Top element: " << stack1.top() << std::endl;
    stack1.push('H');
    std::cout << "Top element: " << stack1.top() << std::endl;

    // Check stack
    std::cout << "\nIs the stack empty? " << stack1.empty() << std::endl;
    std::cout << "Stack size: " << stack1.size() << std::endl;
    std::cout << "Top element: " << stack1.top() << std::endl;

    // Pop
    std::cout << "\nPopping top element..." << std::endl;
    stack1.pop();
    std::cout << "New top element: " << stack1.top() << std::endl;
    std::cout << "Stack size after pop: " << stack1.size() << std::endl;

    // Swap
    std::stack<char> stack2;
    stack2.push('C');
    stack2.push('P');
    stack2.push('E');

    stack1.swap(stack2);

    std::cout << "\nAfter swapping:" << std::endl;
    std::cout << "Stack 1 top element: " << stack1.top() << std::endl;
    std::cout << "Stack 2 top element: " << stack2.top() << std::endl;

    // Emplace
    stack1.emplace('A');
    std::cout << "\nAfter emplacing 'A' onto stack 1, top element: " << stack1.top() << std::endl;

    return 0;
}
```

Operation	

std::stack<char> stack1;	`// Create an object - creating the stack object`
	`std::stack<char> stack1;`
	Creates a stack object named stack1 that stores elements of type char.

empty()	`// isEmpty - checking if the stack is empty`
	`std::cout << "Is the stack empty? " << stack1.empty() << std::endl;`
	Checks whether the stack is empty. Returns 1 (true) if empty and 0 (false) if it contains elements.

push()	`// Push - Adding elements to the stack`
	`stack1.push('R');`
	`std::cout << "Top element: " << stack1.top() << std::endl;`
	`stack1.push('A');`
	`std::cout << "Top element: " << stack1.top() << std::endl;`
	`stack1.push('L');`
	`std::cout << "Top element: " << stack1.top() << std::endl;`
	`stack1.push('P');`
	`std::cout << "Top element: " << stack1.top() << std::endl;`
	`stack1.push('H');`
	Inserts a new element onto the top of the stack.The stack follows the LIFO (Last In, First Out) principle, the most recently pushed element becomes the new top. (H being the top element because it is the last element that got pushed)

top()	`std::cout << "Top element: " << stack1.top() << std::endl;` Returns the value of the top element.
size()	`std::cout << "Stack size: " << stack1.size() << std::endl;` Returns the current number of elements stored in the stack.

pop()	`// Pop`
	`std::cout << "\nPopping top element..." << std::endl;`
	`stack1.pop();`
	Removes the top element from the stack.

swap()	`stack1.swap(stack2);`
	
	`std::cout << "\nAfter swapping:" << std::endl;`
	`std::cout << "Stack 1 top element: " << stack1.top() << std::endl;`
	`std::cout << "Stack 2 top element: " << stack2.top() << std::endl;`
	Exchanges all the contents of one stack with another stack. After swapping, each stack contains the other's original elements.

emplace()	`// Emplace`
	`stack1.emplace('A');`
	`std::cout << "\nAfter emplacing 'A' onto stack 1, top element: " << stack1.top() << std::endl;`
	Constructs and inserts a new element directly at the top of the stack.

OUTPUT:

```
Is the stack empty? 1
Top element: R
Top element: A
Top element: L
Top element: P
Top element: H

Is the stack empty? 0
Stack size: 5
Top element: H

Popping top element...
New top element: P
Stack size after pop: 4

After swapping:
Stack 1 top element: E
Stack 2 top element: P

After emplacing 'A' onto stack 1, top element: A
PS C:\Users\Justine\Desktop\HOA_4p1 (BRAGANZA)>
```

Observation:

Following the creation of the first stack, stack1, I pushed the letters R, A, L, P, and H, forming my name, which is RALPH. As these letters were pushed onto the stack, each letter became the new top of the stack following the Last In, First Out (LIFO) . The letter H became the top of the because it was the last one to be pushed onto the stack. Using the pop() function, the top of the stack which was H was now popped off the stack, making P the new top element and reducing the stack size by one.

Then, I created another stack (stack2) and stored the characters C, P, and E in it. The swap() function was utilized to swap the contents of stack1 and stack2 completely. Therefore, stack1 now held all the elements that were there in stack2. As a result, the top element of stack1 was changed to E. Similarly, stack2 held all the elements from stack1 and, thus, became the top element of stack2. However, H was removed from stack2 by the use of the pop() function. At last, I inserted the character A in stack1 with the help of emplace() function and made it the top element of the stack.

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

CODE:

Braganza_Seatwork4_1.2.cpp

ILOB > Braganza_Seatwork4_1.2.cpp > ...

```cpp
#include <iostream>

// Global Variables
#define maxSize 10
// const int maxSize = 10;

int stackArr[maxSize];
int top = -1, newData;

// Function Prototypes
// Top, Push, Pop, isEmpty, isFull
bool isEmpty();
bool isFull();
void Top();             // The T is capital
void displayAll();      // Goal is to display the stack following the LIFO principle

// Transformer operators
void push();
void pop();

int main() {
    int choice;

    while (true) {
        std::cout << "\n================================================\n";
        std::cout << "Stack Operations\n";
        std::cout << "================================================\n";
        std::cout << "[1] Push\n";
        std::cout << "[2] Pop\n";
        std::cout << "[3] IsEmpty\n";
        std::cout << "[4] IsFull\n";
        std::cout << "[5] Top\n";
        std::cout << "[6] Display All\n";
        std::cout << "================================================\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch (choice) {

            case 1:
                push();
                break;

            case 2:
                pop();
                break;

```

```
            case 3:
                std::cout << "Is the stack empty? " << isEmpty() << std::endl;
                break;

            case 4:
                std::cout << "Is the stack full? " << isFull() << std::endl;
                break;

            case 5:
                Top();
                break;

            case 6:
                displayAll();
                break;

            default:
                std::cout << "Invalid Choice!" << std::endl;
                break;
        }
    }

    return 0;
}

// Function Definitions

bool isEmpty() {
    return (top == -1);
}

bool isFull() {
    return (top == maxSize - 1);
}

void Top() {

    // Error checking
    if (isEmpty()) {
        std::cout << "Stack is empty!" << std::endl;
        return;
    }

    // Print the top element
    std::cout << "The top element is: " << stackArr[top] << std::endl;
}

void push() {

    // Error Checking
    if (isFull()) {
        std::cout << "Stack Overflow!" << std::endl;
        return;
    }

    // Ask the user what they want to insert
    std::cout << "Enter a number to push: ";
    std::cin >> newData;

    // Store the data in the stack
    stackArr[++top] = newData;
}

void pop() {

    // Error Checking
    if (isEmpty()) {
        std::cout << "Stack Underflow!" << std::endl;
        return;
    }

    // Display the item to be popped
    std::cout << "Popped element: " << stackArr[top] << std::endl;

    // Remove the element from the stack
    top--;
}

void displayAll() {

    // Error Checking
    if (isEmpty()) {
        std::cout << "Stack is empty!" << std::endl;
        return;
    }

    // Display the stack following the LIFO principle
    std::cout << "\nStack Contents (Top to Bottom)\n";

    for (int i = top; i >= 0; i--) {
```

```
138         std::cout << stackArr[i] << std::endl;
139     }
140 }
```

MODIFIED CODE:

```
60              case 6:
61                  displayAll();
62                  break;
```

```
126 void displayAll() {
127 
128     // Error Checking
129     if (isEmpty()) {
130         std::cout << "Stack is empty!" << std::endl;
131         return;
132     }
133 
134     // Display the stack following the LIFO principle
135     std::cout << "\nStack Contents (Top to Bottom)\n";
136 
137     for (int i = top; i >= 0; i--) {
138         std::cout << stackArr[i] << std::endl;
139     }
140 }
141 
142 
```

OUTPUT:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 6

Stack Contents (Top to Bottom)
10
9
8
7
6
5
4
3
2
1
```

This is after pushing 10 elements onto the stack (as you can see it follows the LIFO principle because 10 is the Top element because it was the last one in/pushed on to the stack).

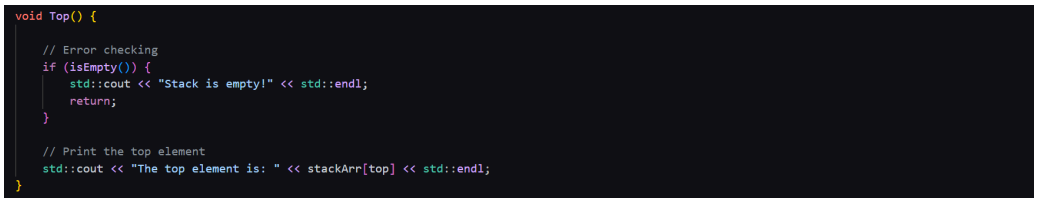
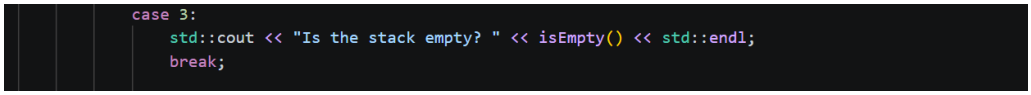
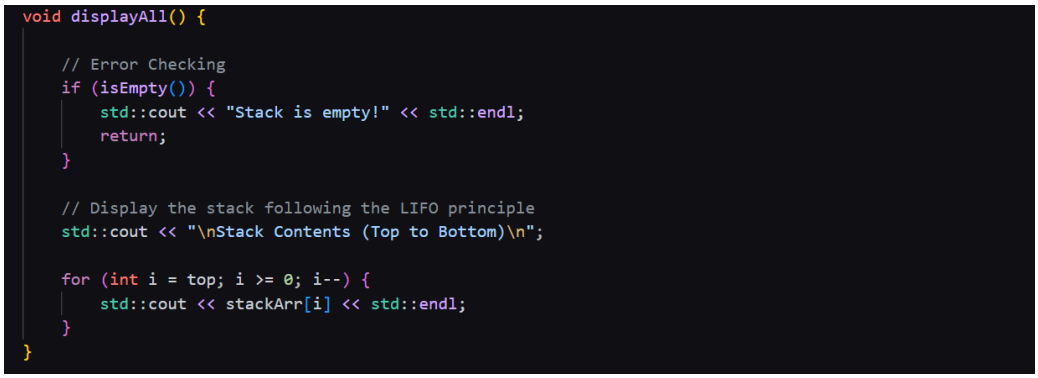
Operation	

MaxSize	`// Global Variables`
	`#define maxSize 10`
	`// const int maxSize = 10;`
	
	`int stackArr[maxSize];`
	`int top = -1, newData;`
	
	The global variables initialize the stack along with all the variables required for initializing the stack. maxSize initializes the capacity of the stack to 10, stackArr stores the data, top variable stores the position of the last item in the stack (initially set to -1 to show an empty stack), and newData stores the input from the user.

push()	`case 1:`
	`push();`
	`break;`
	
	`void push() {`
	`// Error Checking`
	`if (isFull()) {`
	`std::cout << "Stack Overflow!" << std::endl;`
	`return;`
	`}`
	`// Ask the user what they want to insert`
	`std::cout << "Enter a number to push: ";`
	`std::cin >> newData;`
	`// Store the data in the stack`
	`stackArr[++top] = newData;`
	`}`
	
	Adds a new element to the top of the stack. Before inserting, it checks whether the stack is full using isFull(). If there is space, the top is incremented and the new value is stored in the array.

pop()	`case 2:`
	`pop();`
	`break;`
	
	`void pop() {`
	`// Error Checking`
	`if (isEmpty()) {`
	`std::cout << "Stack Underflow!" << std::endl;`
	`return;`
	`}`
	`// Display the item to be popped`
	`std::cout << "Popped element: " << stackArr[top] << std::endl;`
	`// Remove the element from the stack`
	`top--;`
	`}`
	
	Removes the top element from the stack. Before removing, it checks whether the stack is empty using isEmpty(). If the stack contains elements, it displays the popped value and decrements top.

Top()	`case 5:`
	`Top();`
	`break;`

	 Displays the current top element of the stack without removing it. If the stack is empty, an error message is displayed.
isEmpty()	  Checks whether the stack is empty by testing if top is equal to -1. It returns true if the stack is empty and false otherwise.
isFull()	  Checks whether the stack has reached its maximum capacity by testing if top is equal to maxSize - 1. It returns true if the stack is full and false otherwise.
displayAll()	  Displays all the elements in the stack from the top element down to the bottom. It first checks if the stack is empty before displaying the contents.

OUTPUT:

Push:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 1
Enter a number to push: 1
```

Pop and Stack Underflow:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 2
Popped element: 1

==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 2
Stack Underflow!
```

IsEmpty

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 3
Is the stack empty? 1
```

Stack Overflow

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 1
Stack Overflow!

==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 1
Stack Overflow!
```

IsFull

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 4
Is the stack full? 0

==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 1
Enter a number to push: 1

==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 4
Is the stack full? 0
```

Top:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 5
The top element is: 1
```

Display All:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 6

Stack Contents (Top to Bottom)
1
1
1
1
1
1
1
1
1
1
```

B.2. Stacks using Linked Lists

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

CODE:
main.cpp

main.cpp

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout<< "--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---- \n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout<<"\n ------ Popping all the elements ----- \n";

    while(!isEmpty<char>()){
        std::cout<<" Popped: "<< pop<char>() <<std::endl;
    }

    std::cout<<"\n ------ Testing the Underflow ----- \n";
    pop<char>();

    return 0;
}

```

stackList.h

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;        // VALUE THAT WE NEED STORE
    Node* next;   // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T>* head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data= newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

// pop
template<typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout<< "stack overflow. "<<std::endl;
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    // move the head to the next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the removed value
```

```cpp
    return tempVal;
}

// isEmpty
template<typename T>
bool isEmpty(){
    return head <T> == nullptr;
}

// top
template<typename T>
void Top(){
    // error catching:
    if(isEmpty<T>()){
        std::cout<<"Stack underflow "<<std::endl;
        return;
    }
    // display the Top node
    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
}

//display All
template <typename T>
void displayALL(){
    if (head<T> == nullptr){
        std::cout << "stack undeflow\n";
        return;
    }

    std::cout<< "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while(temp != nullptr){
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout << "\n";
}

#endif
```

OUTPUT:

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ----
Top of Stack: e
Displaying the stack:
e
p
c


 ------ Popping all the elements -----
 Popped: e
 Popped: p
 Popped: c

 ------ Testing the Underflow -----
stack overflow.
```

MODIFIED CODE:

```
69  //display All
70  template <typename T>
71  void displayALL(){
72      if (head<T> == nullptr){
73          std::cout << "stack undeflow\n";
74          return;
75      }
76
77      std::cout<< "Displaying the stack: \n";
78      Node<T>* temp = head<T>;
79      while(temp != nullptr){
80          std::cout << temp->data << "\n";
81          temp = temp->next;
82      }
83      std::cout << "\n";
84  }
85
86  #endif
```

Operations	

Push()	main.cpp
	`6 std::cout<< "--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---- \n";`
	`7 //push`
	`8 push('c');`
	`9 push('p');`
	`10 push('e');`
	stackList.h
	`18 // push`
	`19 template <typename T>`
	`20 void push(T newData){`
	`21 //create a new node and assign the newData`
	`22 Node<T>* newNode = new Node<T>;`
	`23 newNode->data= newData;`
	`24`
	`25 //new node points to current top, then becomes the new top`
	`26 newNode->next = head<T>;`
	`27 head<T> = newNode;`
	`28 }`
	Description:
	Creates a new node, stores the given data in it, and places it at the top of the stack by updating the head pointer.

Pop()	main.cpp
	`21 std::cout<<"\n ------ Testing the Underflow ----- \n";`
	`22 pop<char>();`
	stackList.h

	``cpp
	// pop
	template<typename T>
	T pop(){
	    //check if the stack is empty:
	    if(head<T> == nullptr){
	        std::cout<< "stack overflow. "<<std::endl;
	    }
	
	    //save the current top node
	    Node<T>* temp = head<T>;
	    //store its data
	    T tempVal = temp->data;
	
	    // move the head to the next node
	    head<T> = head<T>->next;
	    //free the removed node
	    delete temp;
	    //return the removed value
	    return tempVal;
	}
	``
	
	Description:
	Removes the node at the top of the stack, returns its value, updates the head pointer to the next node, and frees the removed node's memory.

Top()	main.cpp
	``cpp
	    //display the current top
	    Top<char>(); // Top should be 'e'
	``
	stackList.h
	
	``cpp
	// top
	template<typename T>
	void Top(){
	    // error catching:
	    if(isEmpty<T>()){
	        std::cout<<"Stack underflow "<<std::endl;
	        return;
	    }
	    // display the Top node
	    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
	}
	``
	
	Description:
	Displays the value stored in the top node without removing it from the stack. If the stack is empty, it displays an underflow message.

isEmpty()	main.cpp
	``cpp
	    while(!isEmpty<char>()){
	        std::cout<<" Popped: "<< pop<char>() <<std::endl;
	    }
	``
	stackList.h
	``cpp
	// isEmpty
	template<typename T>
	bool isEmpty(){
	    return head <T> == nullptr;
	}
	``
	
	Description:
	Checks whether the stack is empty by determining if the head pointer is nullptr. Returns true if empty and false otherwise.

displayALL()	main.cpp

```
14      displayALL<char>();
15      std::cout<<"\n ------ Popping all the elements ----- \n";
```

stackList.h

```
69  //display All
70  template <typename T>
71  void displayALL(){
72      if (head<T> == nullptr){
73          std::cout << "stack undeflow\n";
74          return;
75      }
76
77      std::cout<< "Displaying the stack: \n";
78      Node<T>* temp = head<T>;
79      while(temp != nullptr){
80          std::cout << temp->data << "\n";
81          temp = temp->next;
82      }
83      std::cout << "\n";
84  }
```

Description:
Traverses the linked list from the top node to the last node, displaying every element.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:
1. Create a Stack.
2. While(end of input is not reached) {
    a) If the character read is not a symbol to be balanced, ignore it.
    b) If the character is an opening symbol, push it onto the stack.
    c) If it is a closing symbol:
        i) Report an error if the stack is empty.
        ii) Otherwise, pop the stack.
    d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

CODE: Stack using Arrays and Stack using Linked Lists

main.cpp

main.cpp ● C theBalancedStack.h

ILOC3 > main.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "theBalancedStack.h"

int main() {

    std::string input;
    std::cout << "-------SYMBOL BALANCER-------\n";

    std::cout << "Enter an expression: ";
    std::getline(std::cin, input);


    // Results Using Array Stack
    std::cout << "\nArray Stack: ";

    if (checkArray(input))
        std::cout << "VALID\n";
    else
        std::cout << "INVALID\n";


    // Results Using Linked List Stack
    std::cout << "Linked List Stack: ";

    if (checkList(input))
        std::cout << "VALID\n";
    else
        std::cout << "INVALID\n";


    return 0;
}

```

theBalancedStack.h

main.cpp ● C theBalancedStack.h ×

ILOC3 > C theBalancedStack.h > ...

```cpp
#ifndef THEBALANCEDSTACK_H
#define THEBALANCEDSTACK_H

#include <iostream>
#include <string>


/*

Array Stack Implementation

*/

char stackArr[100];
int top = -1;

// Push data into array stack
void pushArr(char data) {
    if (top == 99)
        return;
    stackArr[++top] = data;
}

// Remove top element
char popArr() {
    if (top == -1)
        return '\0';
    return stackArr[top--];
}
```

```
30
31  // Check if array stack is empty
32  bool emptyArr() {
33      return top == -1;
34  }
35
36  // Reset array stack
37  void resetArr() {
38      top = -1;
39  }
40
41  // Balanced symbol checking using array stack
42  bool checkArray(std::string input) {
43      resetArr();
44      for (int i = 0; i < input.length(); i++) {
45          char symbol = input[i];
46          // If opening symbols, push into stack
47          if (symbol == '(' || symbol == '{' || symbol == '[') {
48              pushArr(symbol);
49          }
50          // If closing symbols, check the stack
51          else if (symbol == ')' || symbol == '}' || symbol == ']') {
52              if (emptyArr())
53                  return false;
54              char open = popArr();
55              if ((symbol == ')' && open != '(') ||
56                  (symbol == '}' && open != '{') ||
57                  (symbol == ']' && open != '[')) {
58                  return false;
59              }
60          }
61      }
62      return emptyArr();
63  }
64
65  /*
66
67  Linked List Stack Implementation
68
69  */
70
71  struct Node {
72      char data;
73      Node* next;
74  };
75
76  Node* head = nullptr;
77
78  // Push into linked list stack
79  void pushLL(char data) {
80      Node* newNode = new Node;
81      newNode->data = data;
82      newNode->next = head;
83      head = newNode;
84  }
85
86  // Remove top node
87  char popLL() {
88      if (head == nullptr)
89          return '\0';
90      Node* temp = head;
91      char value = temp->data;
92      head = head->next;
93      delete temp;
94      return value;
95  }
96
97  // Check if linked list stack is empty
98  bool emptyLL() {
99      return head == nullptr;
100 }
101
102 // Clear linked list stack
103 void resetLL() {
104     while (!emptyLL()) {
105         popLL();
106     }
107 }
108
```

```
// Balanced symbol checking using linked list
bool checkList(std::string input) {
    resetLL();
    for (int i = 0; i < input.length(); i++) {
        char symbol = input[i];
        // Opening symbols
        if (symbol == '(' || symbol == '{' || symbol == '[') {
            pushLL(symbol);
        }
        // Closing symbols
        else if (symbol == ')' || symbol == '}' || symbol == ']') {
            if (emptyLL())
                return false;
            char open = popLL();
            if ((symbol == ')' && open != '(') ||
                (symbol == '}' && open != '{') ||
                (symbol == ']' && open != '[')) {
                return false;
            }
        }
    }

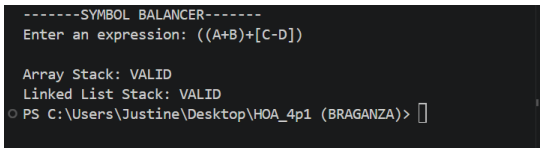
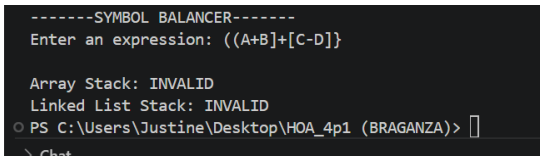

    return emptyLL();
}


#endif
```

Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Consolel Screenshot)	Analysis
(A+B)+(C-D)	Y	-------SYMBOL BALANCER------- Enter an expression: (A+B)+(C-D) Array Stack: VALID Linked List Stack: VALID PS C:\Users\Justine\Desktop\HOA_4p1 (BRAGANZA)>	As the program reads the expression, every opening parenthesis " (" is pushed into the stack using pushArr() or pushLL(). When a closing parenthesis) is encountered, popArr() or popLL() removes the top " (" and compares it with ") ". Since all pairs match and the stack is empty at the end (emptyArr()/emptyLL() returns true), the expression is **VALID.**
((A+B)+(C-D)	N	-------SYMBOL BALANCER------- Enter an expression: ((A+B)+(C-D) Array Stack: INVALID Linked List Stack: INVALID PS C:\Users\Justine\Desktop\HOA_4p1 (BRAGANZA)>	The program pushes two opening parentheses " (" onto the stack, but only one of them is removed because the expression is missing a closing ") ". After the loop finishes, emptyArr() and emptyLL()

			return false because one unmatched " (" is still in the stack, so the expression is **INVALID.**
((A+B)+[C-D])	Y		The program pushes both opening parentheses " (" and the opening square bracket " [" onto the stack. Each closing symbol " (] " and ")) " causes the top opening symbol to be popped and compared. Since every popped symbol matches its corresponding closing symbol, the stack becomes empty at the end, making the expression **VALID.**
((A+B]+[C-D]}	N		When it encounters the first closing square bracket "] ", popArr() or popLL() removes an opening parenthesis " (" from the top of the stack. The comparison symbol == ']' && open != '[' evaluates to true, causing the function to immediately return false. Because the symbols do not match, the expression is **INVALID.**

CODE OVERVIEW:

The program determines if an expression has matched symbols by utilizing two types of stack implementations: an array stack and a linked list stack implementation. The expression is read character by character, where every opening symbol, either (, {, or [, is pushed into the stack while any closing symbol), }, or] triggers the popping of the top symbol in the stack. Then, the popped symbol will be checked to see if it matches any of the closing symbols. If a mismatch is found, the stack is empty when a closing symbol appears, or there are still opening symbols left in the stack after processing the entire expression, the program reports the expression as INVALID. Otherwise, the expression is marked as VALID.

CODE: Stack using C++ STL (OPTIONAL)

main.cpp

ILOC > main.cpp > ...

```cpp
#include <iostream>
#include <stack>
#include <string>

// Function to check if the opening and closing symbols match
bool checkPair(char open, char close) {

    if (open == '(' && close == ')')
        return true;

    if (open == '[' && close == ']')
        return true;

    if (open == '{' && close == '}')
        return true;

    // Symbols do not match
    return false;
}

// Function to check if an expression is balanced
bool checkBalance(std::string exp) {

    // Create a stack to store opening symbols
    std::stack<char> st;

    // Read each character in the expression
    for (int i = 0; i < exp.length(); i++) {

        char ch = exp[i];

        // If the character is an opening symbol, push it onto the stack
        if (ch == '(' || ch == '[' || ch == '{') {
            st.push(ch);
        }

        // If the character is a closing symbol
        else if (ch == ')' || ch == ']' || ch == '}') {

            // If the stack is empty, there is no matching opening symbol
            if (st.empty()) {
                return false;
            }

            // Get the top opening symbol
            char temp = st.top();

            // Remove it from the stack
            st.pop();

            // Check if the symbols match
            if (!checkPair(temp, ch)) {
                return false;
            }
        }

        // Ignore letters, numbers, operators, and spaces
    }

    // If the stack is empty, all symbols were matched
    if (st.empty())
        return true;
    else
        return false;
}

int main() {

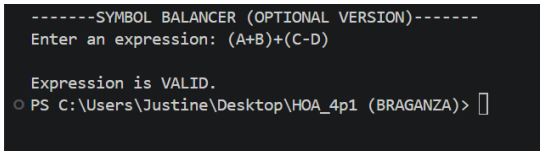
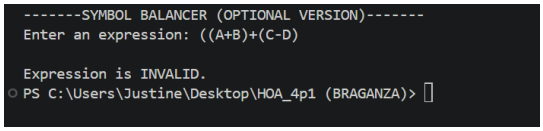
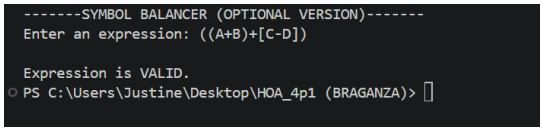
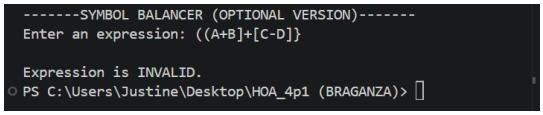
    // Variable to store the user's expression
    std::string exp;

    // Ask the user for an expression
    std::cout << "-------SYMBOL BALANCER (OPTIONAL VERSION)-------\n";
    std::cout << "Enter an expression: ";
    std::getline(std::cin, exp);

    // Check if the expression is balanced
    if (checkBalance(exp))
        std::cout << "\nExpression is VALID.\n";
    else
        std::cout << "\nExpression is INVALID.\n";

    return 0;
}
```

Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Consolel Screenshot)	Analysis
(A+B)+(C-D)	Y		The "(" before the A+B expression matches the ")" in the beginning, while the "(" before the C-D expression matches the ")" that comes next. All parentheses have been correctly balanced, so the stack is empty.
((A+B)+(C-D)	N		The expression is missing one closing ")". The first opening "(" at the beginning of the expression is never matched, leaving one unmatched "(" in the stack.
((A+B)+[C-D])	Y		The expression has matching parentheses and brackets. Both opening "(" symbols are matched with ")" symbols, and the opening "[" is correctly matched with "]", leaving the stack empty.
((A+B]+[C-D]}	N		The expression contains mismatched symbols. The closing "]" attempts to match an opening "(" instead of "[", and the closing "}" does not match the remaining opening "(", causing the expression to be invalid.

Conclusion/Learnings

After performing this hands-on activity, I learned how stacks can be used for checking whether symbols in an expression are balanced and how the same concept can be applied to other projects in different ways. I also learned the differences between implementing a stack using an array and using a linked list. Although both implementations follow the same LIFO principle, they differ in how memory is managed and how elements are stored. Despite the difficulties, performing ILOA, ILOB, and ILOB2 helped a lot with referencing exactly what I needed to do to be able to perform the supplementary activity.

The procedure helped me understand the importance of the push, pop, and empty operations in solving the balanced-symbol problem because without those first procedures, I don't think I will be able to understand it. By processing one character at a time, pushing opening symbols, and popping and comparing them with closing symbols, I was able to see how stacks simplify expression validation. Implementing both array and linked-list versions also reinforced my understanding of their similarities and differences. Making the stack of arrays was quite simple and straightforward to make; it was still difficult, of course, but not as difficult as the linked list because I still lack knowledge in that department.

The supplementary activity gave me the opportunity to test different expressions and observe how the program handled valid and invalid cases. It helped me understand why expressions fail, such as when symbols are mismatched or when opening symbols remain in the stack after processing the entire expression. It was easier for me to confirm the functionality of my program since I have gone through a lot of errors while debugging these cases. I have come to know about the return '0\'. It is done to return the null character if there is no character at all in the string, for example, popping an element from an empty stack.

Overall, I think I completed the activity successfully despite the hours of struggle and burnouts from not understanding it at first; I've gained a better understanding of stack implementations and their practical applications. My areas for improvement include writing cleaner and more efficient code because I used AI to help me with my code formatting. Improving my debugging skills to be able to code faster but still code proper code and becoming more comfortable implementing data structures without relying heavily on references and searching online for guides.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini and QuillBot and Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, summarization, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: AI was used for stack concepts, explain the balanced-symbol algorithm (I did not understand the instructions at first), assist in debugging compile errors, and provide guidance on improving code structure. The final implementation, testing, analysis, and submission were reviewed, modified, and completed by me. Most of the code was referenced and reused from ILOA, ILOB and ILOB2 as a framework for the supplementary activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.